



Combine

Lecture 1 Software Engineering

1. 软件工程（Software Engineering）

定义:

软件工程是将工程原则应用于软件系统的开发、操作和维护的过程，以确保高质量和高效率。

关键组成部分:

- **流程（Processes）**:系统化的活动序列，包括规划、分析、设计、实施、测试和维护。
- **工具（Tools）**:用于支持流程和方法的工具，例如IDE、版本控制系统、测试框架。
- **方法论（Methodologies）**:指导如何实现流程的框架，例如Agile、Waterfall、DevOps。

2. 软件开发生命周期（Software Development Lifecycle, SDLC）

主要阶段:

- 规划（Planning）
- 分析（Analysis）
- 设计（Design）
- 实施（Implementation）
- 维护（Maintenance）

3. 流程的重要性（Importance of Processes）

- **确保一致性和质量（Consistency and Quality）**:标准化流程能保持团队间的一致性并提高软件质量。

- **管理复杂性和降低风险 (Manage Complexity and Reduce Risks)** :通过分解复杂系统,降低潜在风险。
- **促进沟通 (Facilitate Communication)** :为利益相关者提供统一的框架和术语。

4. 软件流程与需求的关系 (Relationship Between Software Processes and Requirements)

- **基础性 (Foundation)** :软件需求决定了开发范围,并对后续阶段(设计、测试等)提供指导。
- **差的需求影响 (Impact of Poor Requirements)** :不明确或错误的需求会导致开发问题、成本增加或项目失败。
- **改进需求质量的流程:**
 - **需求获取 (Requirement Elicitation)**
 - **需求验证 (Requirement Validation)**
 - **需求管理 (Requirement Management)**

5. 需求工程 (Requirements Engineering)

定义:

需求工程是一个系统化的过程,用于识别、记录和维护软件需求,确保软件符合用户需求和业务目标。

关键活动:

- **获取 (Elicitation)**
- **文档化 (Documentation)**
- **验证 (Validation)**
- **管理 (Management)**

6. 需求工程的重要性 (Importance of Requirements Engineering)

- **防止范围蔓延 (Prevent Scope Creep)**
- **降低开发成本与时间 (Reduce Development Costs and Time)**
- **提高软件质量与用户满意度 (Improve Software Quality and User Satisfaction)**

- 最小化风险 (Minimize Risks)
- 确保合规性 (Ensure Compliance)

7. 需求工程的挑战 (Challenges in Requirements Engineering)

- 需求变化 (Changing Requirements)
- 模糊性与误解 (Ambiguities and Misunderstandings)
- 沟通差距 (Communication Gaps)
- 技术可行性问题 (Technical Feasibility Issues) 解决方案:
- 使用清晰的语言和可视化工具。
- 持续与利益相关者沟通。
- 使用需求管理工具。

8. MoSCoW 方法 (MoSCoW Method)

- **Must Have:** 必须满足的关键需求。
- **Should Have:** 高优先级但可推迟的需求。
- **Could Have:** 可选需求。
- **Won't Have:** 当前阶段不会实现的需求。

9. 需求分析与文档 (Analysis and Specification)

- 需求分析: 理解、优化和优先化需求，例如使用**MoSCoW**方法。
- 文档化 (Specification) : 通过用例图 (Use Case Diagrams)、用户故事 (User Stories) 和SRS等格式记录需求。

10. 验证 (Validation) 与管理 (Management)

- 验证: 确保需求完整、可行、一致。
- 管理: 随着项目变化不断更新和追踪需求。

总结

需求工程是软件开发的核​​心，其通过系统化的获取、分析、验证和管理流程，确保软件符合用户需求和业务目标。在整个软件开发生命周期中，需求工程不仅起到连接利益相关者与开发者的桥梁作用，还能降低开发风险、提高软件质量。通过MoSCoW方法和清晰的文档化流程，需求工程帮助团队更高效地实现目标，最终交付满足需求的高质量软件。

Lecture 2 软件需求工程 (Software Requirements Engineering)

1. 需求获取的初始阶段 (Requirements Inception)

目标 (Objectives):

- 识别关键利益相关者 (Key Stakeholders)。
 - 理解业务需求 (Business Needs) 和目标。
 - 定义系统的高层范围 (High-Level System Scope)。
 - 产出 (Outcome): 明确需要解决的问题。
-

2. 领域理解 (Domain Understanding)

目标 (Objectives):

- 了解 行业、业务流程 (Business Processes) 及面临的挑战。
- 掌握 领域专属术语 (Domain-Specific Terminology)。

方法 (How to Achieve It):

- ✓ 研究 (Studying) 现有系统和文档。
 - ✓ 访谈 (Interviewing) 领域专家。
 - ✓ 观察 (Observing) 真实业务操作。
-

3. 产品愿景 (Product Vision)

定义 (Definition):

产品愿景是对产品目标的高层描述 (A high-level statement describing what the product will achieve)。

关键要素 (Key Elements):

1. 目标用户 (Target Users)
2. 业务价值 (Business Value)
3. 竞争优势 (Competitive Advantage)

✅ 示例 (Example):

"An AI-powered chatbot to automate customer service and reduce response time by 50%."

4. 定义范围 (Defining the Scope)

目标 (Objectives):

- 确定 系统包含 (Included) 和不包含 (Excluded) 的功能。
- 避免 范围蔓延 (Scope Creep)。

范围声明组件 (Scope Statement Components):

1. 系统功能 (System Features)
2. 边界 & 约束 (Boundaries & Constraints)
3. 假设 (Assumptions)

✅ 示例 (Example):

"The system will allow users to register, log in, and make purchases, but will not handle inventory management."

5. 需求获取 (Requirements Elicitation)

定义 (Definition):

需求获取是从利益相关者处收集需求的过程 (The process of gathering requirements from stakeholders)。

面临的挑战 (Challenges):

- ✗ 业务需求不清晰 (Unclear Business Needs)
- ✗ 利益相关者需求冲突 (Conflicting Stakeholder Interests)
- ✗ 需求表达困难 (Difficulty in Expressing Requirements)

6. 访谈 (Interview Technique)

定义 (Definition):

访谈是一种 对话式方法 (Conversational Method)，让利益相关者分享他们的需求和期望。

访谈类型 (Types of Interviews):

1. 结构化访谈 (Structured Interview) - 预设问题，保证一致性。
2. 半结构化访谈 (Semi-Structured Interview) - 预设框架+灵活提问。
3. 非结构化访谈 (Unstructured Interview) - 开放式对话，深度探索。

✔ 优缺点 (Benefits & Limitations):

访谈类型	优势	限制
结构化 (Structured)	结果一致，易于比较	限制深入探索
半结构化 (Semi-Structured)	平衡一致性与灵活性	仍受框架限制
非结构化 (Unstructured)	适合深入挖掘	结果难以比较

访谈准备 (Preparing for an Interview):

1. 明确访谈目标 (Define Interview Objectives)
2. 确定受访者 (Identify Interviewees)
3. 准备关键问题 (Prepare Questions to Cover Key Areas of Inquiry)

访谈执行 (Conducting an Interview):

1. 介绍 (Introduction)
 2. 提问 (Ask Questions)
 3. 鼓励讨论 (Encourage Discussion)
 4. 记录回应 (Record Responses for Later Analysis)
-

7. 问卷调查 (Questionnaire Technique)

定义 (Definition):

问卷是一个 **结构化表单 (Structured Form)**，让利益相关者填写需求信息。

问卷类型 (Types of Questionnaires):

1. **开放式 (Open-Ended)** - 允许受访者自由回答，获取深入信息。
2. **封闭式 (Close-Ended)** - 预设选项，方便数据分析。

✅ 示例 (Example):

开放式问题 (Open-Ended Question):

"What challenges do you face when using this system, and how do you think it could be improved?"

封闭式问题 (Close-Ended Question):

"How satisfied are you with the system's ease of use?"

- Very Satisfied
- Satisfied
- Neutral
- Dissatisfied
- Very Dissatisfied

✅ 优缺点 (Benefits & Limitations):

类型	优势	限制
开放式 (Open-Ended)	深入反馈	难以量化
封闭式 (Close-Ended)	便于分析	信息有限

8. 观察法 (Observation Technique)

定义 (Definition):

通过 **观察用户交互 (Watching Users Interact)** 来获取真实需求。

观察类型 (Types of Observation):

1. **直接观察 (Direct Observation)** - 观察用户，但不干涉。
2. **参与式观察 (Participant Observation)** - 观察者直接参与工作环境。

✓ 优缺点 (Benefits & Limitations):

类型	优势	限制
直接观察 (Direct)	真实用户行为	观察者可能有偏见
参与式观察 (Participant)	深入理解	可能影响用户行为

9. 文档阅读 (Document Reading Technique)

定义 (Definition):

通过 **分析现有文档 (Reviewing Existing Documents)** 理解系统需求。

常见文档类型 (Types of Documents Used):

- 用户手册 (Manuals)
- 技术规格 (Technical Specifications)
- 历史记录 (Historical Records)

✓ 优缺点 (Benefits & Limitations):

优势	限制
提供历史背景	可能过时
不需要利益相关者参与	缺乏互动

10. 需求文档 (Requirements Documentation)

作用 (Purpose):

- 📌 开发和测试的参考 (Reference for Development & Testing)
- 📌 防止沟通误解 (Prevents Miscommunication)

需求文档类型 (Types of Requirement Documents):

1. 软件需求规格 (Software Requirements Specification, SRS)
2. 用例描述 (Use Case Descriptions)
3. 用户故事 (User Stories)

✅ 良好需求的特征 (Characteristics of Good Requirements):

- ✓ 清晰 (Clear)
- ✓ 完整 (Complete)
- ✓ 一致 (Consistent)
- ✓ 可行 (Feasible)

✅ 示例 (Example):

"The system shall allow users to reset their passwords via email verification."

总结 (Final Summary)

- 需求获取 (Requirements Elicitation) 包括访谈、问卷、观察、文档阅读等方法。
- 产品愿景 (Product Vision) 和 需求范围 (Scope Definition) 确保目标明确。
- 良好的需求文档 (Requirements Documentation) 保障开发顺利进行。

✅ 掌握软件需求工程，确保项目成功！🚀

Lecture 3 软件需求分析、建模与规格说明 (Requirements Analysis, Modeling, and Specification)

1. 学习目标 (Learning Objectives)

- 理解 **描述 (Description)** 与 **规格说明 (Specification)** 之间的区别。
 - 学习各种 **建模技术 (Modeling Techniques)** 和 **图示表示法 (Diagrammatic Notations)**。
 - 探索 **用例建模 (Use Case Modeling)** 和 **场景描述 (Scenario Descriptions)**。
 - 深入了解 **领域分析 (Domain Analysis)** 与 **需求理解 (Requirements Understanding)**。
 - 介绍 **形式化规格说明 (Formal Specification)** 技术。
-

2. 需求分析简介 (Introduction to Requirements Analysis)

定义 (Definition):

需求分析是 **确定用户对新系统或修改系统的期望 (Process of determining user expectations for a new or modified system)** 的过程。

重要性 (Importance):

- ✓ 防止 **需求误解 (Requirement Misunderstandings)**。
 - ✓ 确保 **更好的系统设计 (Better System Design)**。
 - ✓ 降低 **项目失败风险 (Reduces Project Failure Risk)**。
-

3. 描述 vs. 规格说明 (Description vs. Specification)

描述 (Description)

- **自然语言 (Natural Language)** 解释系统行为。
- 通常是**非正式 (Informal)** 且容易产生歧义 (**Ambiguous**)。

规格说明 (Specification)

- **精确 (Precise)** 且**结构化 (Structured)**，清晰定义系统行为。
- 使用**模型 (Models)**、**图表 (Diagrams)** 和**形式化符号 (Formal Notations)**。

✓ **示例 (Example):**

类型	示例
描述 (Description)	"The system should allow users to login."
规格说明 (Specification)	"The system shall authenticate users using a valid username and password stored in the database."

4. 需求建模技术 (Requirements Modeling Techniques)

建模的目的 (Purpose of Modeling):

- 📌 可视化 (Visual Representation) 复杂系统需求。
- 📌 提高理解 (Enhances Understanding) 和沟通 (Communication)。

关键技术 (Key Techniques):

1. 用例建模 (Use Case Modeling)
2. 数据流图 (Data Flow Diagram, DFD)
3. 实体-关系图 (Entity-Relationship Diagram, ERD)
4. 类图 (Class Diagram, UML)

5. 用例建模与场景描述 (Use Case Modeling and Scenario Descriptions)

用例图 (Use Case Diagram):

- 捕捉用户与系统的交互 (Captures interactions between users and the system)。
- 确定系统功能 (Identifies system functionality)。

场景描述 (Scenario Descriptions):

- 不同路径 (Different Paths) 在系统操作中的情况：
 1. 正常流程 (Normal Flow)
 2. 备选流程 (Alternative Flow)
 3. 异常流程 (Exception Flow)

✅ 示例 (Example):

用例名称 (Use Case Name)	登录流程 (Login Process)
参与者 (Actors)	用户 (User), 认证系统 (Authentication System)
前置条件 (Preconditions)	用户必须已有账号
主流程 (Main Flow)	用户输入用户名和密码 → 系统验证 → 允许访问
备选流程 (Alternative Flow)	输入错误密码 → 显示错误信息

6. 数据流图 (Data Flow Diagram, DFD)

定义 (Definition): 表示数据如何在系统中流动 (Represents how data moves through a system)。

关键组件 (Components):

- ◆ 处理 (Processes) - 圆形 (Circle)
- ◆ 数据存储 (Data Stores) - 矩形 (Rectangle)
- ◆ 外部实体 (External Entities) - 正方形 (Square)
- ◆ 数据流 (Data Flows) - 箭头 (Arrows)

7. 实体-关系图 (Entity-Relationship Diagram, ERD)

定义 (Definition): 用于建模数据库结构 (Diagram that models database structure)。

关键元素 (Key Elements):

- 实体 (Entities) - 例如：学生 (Student), 课程 (Course)
- 属性 (Attributes) - 例如：学生姓名 (Student Name), 课程代码 (Course Code)
- 关系 (Relationships) - 例如："注册于 (Enrolled-In)"

8. 领域分析与需求理解 (Analysis for Understanding the Domain and Requirements)

领域分析 (Domain Analysis):

📌 识别 **系统环境 (System Environment)**、**利益相关者 (Stakeholders)** 和 **约束 (Constraints)**。

📌 **技术 (Techniques)**: 访谈 (Interviews), 问卷调查 (Questionnaires), 文档分析 (Document Analysis)。

需求分析方法 (Requirement Analysis Approaches):

✓ **结构化分析 (Structured Analysis)** - 使用 DFD, ERD。

✓ **面向对象分析 (Object-Oriented Analysis)** - 使用 **UML 图 (UML Diagrams)**, 类图 (Class Diagrams)。

9. 形式化规格说明 (Formal Specification)

定义 (Definition):

📌 采用**数学方法 (Mathematically-based Techniques)** 来精确地定义软件需求。

优点 (Why is it useful?):

✓ **消除歧义 (Removes Ambiguity)**

✓ **提高验证和确认 (Improves Verification and Validation)**

常见方法 (Common Methods):

1. Z-Notation

2. B-Method

3. VDM (Vienna Development Method)

✓ **优势 (Benefits of Using Formal Specification):**

✓ **精准 (Precise) 且无歧义 (Unambiguous)**

✓ **减少误解 (Reduces Misinterpretation)**

✓ **在开发前帮助系统验证 (Helps with System Validation Before Development)**

10. 需求分析的挑战 (Challenges in Requirements Analysis)

常见问题 (Common Issues):

- ✗ 需求不明确 (Unclear or Vague Requirements)
- ✗ 需求不断变化 (Changing Requirements)
- ✗ 利益相关者需求冲突 (Conflicting Stakeholder Interests)

解决方案 (How to Overcome Them?):

- ✓ 与利益相关者定期沟通 (Regular Communication with Stakeholders)
 - ✓ 使用原型 (Prototypes) 和可视化模型 (Visual Models)
-

11. 真实案例：软件需求失败 (Real-World Example – Software Requirement Failure)

案例研究 (Case Study): 伦敦救护系统 (London Ambulance System Failure)

📌 失败原因 (Failure Reasons):

- ✗ 缺乏明确需求 (System designed without clear requirements)
 - ✗ 分析不足导致错误调度 (Lack of proper analysis led to incorrect dispatching)
 - ✓ 教训 (Lesson): 需求工程 (Requirement Engineering) 重要性！
-

12. 行业最佳实践 (Industry Best Practices in Requirements Engineering)

✓ 定义 SMART 需求 (Define SMART Requirements):

- Specific (具体)
- Measurable (可衡量)
- Achievable (可实现)
- Relevant (相关)
- Time-bound (有时间限制)

✓ 使用需求管理工具 (Use Requirement Management Tools):

- JIRA
 - IBM Rational DOORS
 - ReqSuite
-

总结 (Final Summary)

- 📌 需求分析 (Requirement Analysis) 确保需求清晰、完整、正确。
 - 📌 建模技术 (Modeling Techniques) 通过可视化方式提高需求理解。
 - 📌 用例建模 (Use Case Modeling) 记录用户交互。
 - 📌 形式化规格说明 (Formal Specification) 通过数学方法精确定义需求。
 - ✅ 掌握需求分析，确保项目成功！🚀
-

Lecture 4: 软件验证与确认 (Software Verification and Validation, V&V)

1. 学习目标 (Learning Objectives)

- 理解 验证 (Verification) 与 确认 (Validation) 之间的区别。
 - 探索需求 V&V 技术 (Techniques for Requirements Verification & Validation)。
 - 学习如何 识别冲突 (Identify Conflicts)、发现不一致 (Detect Inconsistencies) 和检查需求完整性 (Check for Completeness)。
-

2. 什么是需求 V&V ? (What is Requirements Verification & Validation?)

✅ 验证 (Verification)

- 📌 检查需求文档是否正确 (Check if the requirement document is correct and logically structured)。
- 📌 关注“我们是否正确地构建产品？” (Are we building the product right?)

✅ 确认 (Validation)

📌 确保需求真实反映了利益相关者的需求 (Confirm if the documented requirements truly reflect the needs of stakeholders)。

📌 关注“我们是否构建了正确的产品？” (Are we building the right product?)

3. V&V 的重要性 (Importance of V&V)

✓ 减少后期昂贵的返工 (Reduces costly rework in later stages)

✓ 提高客户满意度 (Ensures customer satisfaction)

✓ 改善软件质量 (Improves software quality)

✓ 案例 (Example): NASA 曾因需求错误损失 \$1.25 亿的航天器 (NASA lost a \$125 million spacecraft because of a requirements error)。

4. V&V 发生的时间 (When Does V&V Happen?)

📌 贯穿整个软件开发生命周期 (Throughout the Software Development Life Cycle, SDLC)

📌 特别是在需求获取 (Requirements Elicitation) 和规格说明 (Specification) 期间

📌 在设计 (Design) 和实现 (Implementation) 之前

5. 详细解析验证与确认 (In-Depth Verification & Validation)

✅ 验证 (Verification) 关注点：

✓ 确保软件符合 指定需求 (Meets Specified Requirements)

✓ 关注 正确性 (Correctness) 和 一致性 (Consistency)

✓ 技术 (Techniques): 评审 (Reviews), 走查 (Walkthroughs), 检查 (Inspections)

✅ 确认 (Validation) 关注点：

✓ 确保软件 符合用户需求和期望 (Fulfills User Needs and Expectations)

✓ 涉及 利益相关者 (Stakeholders)

✓ **技术 (Techniques):** 原型 (Prototyping), 用户反馈 (User Feedback), 验收测试 (Acceptance Testing)

6. V&V 之间的关系 (The V&V Relationship)

◆ 验证 (Verification) = 内部检查 (Internal Check)

◆ 确认 (Validation) = 外部检查 (External Check)

✓ 两者缺一不可，否则会导致灾难性后果 (Both are essential; one without the other can lead to disaster)。

7. 需求阶段的 V&V 活动 (V&V Activities in Requirements Phase)

📌 需求评审 (Requirements Review)

📌 一致性检查 (Consistency Checks)

📌 使用原型进行确认 (Prototyping for Validation)

📌 可追溯性分析 (Traceability Analysis)

8. 需求 V&V 技术 (Inspection Techniques in V&V)

✓ **检查 (Inspection) 技术：**

📌 正式 (Formal)、基于同行 (Peer-Based) 的文档检查

📌 使用检查表 (Checklist-Driven) 关注：

✓ 歧义 (Ambiguities)

✓ 冲突 (Conflicts)

✓ 不完整性 (Incompleteness)

✓ **走查 (Walkthroughs)：**

📌 非正式 (Informal) 讨论

📌 由演示者 (Presenter) 讲解，团队提供反馈

📌 目标：理解和评估需求 (Goal: Understand and Evaluate the Requirements)

✅ 评审 (Reviews) :

- 📌 技术评审会议 (Technical Review Meetings)
 - 📌 用于早期发现错误 (Detect Errors Early)
 - 📌 可以是正式 (Formal) 或非正式 (Informal)
 - 📌 示例 (Example): 每周与客户或 QA 团队进行需求评审 (Weekly review sessions with clients or internal QA team)
-

9. 原型验证 (Prototyping for Validation)

- 📌 构建系统或界面的模型 (Build a mock-up of the system or interface)
 - 📌 收集用户反馈 (Gather feedback from users)
 - 📌 发现隐藏需求 (Helps discover hidden needs)
 - 📌 因为“等我看到它就知道了” (I'll know it when I see it) 不是一个明确的需求！
-

10. 验收标准与测试 (Acceptance Criteria and Testing)

- 📌 定义“需求完成”的标准 (Define what it means for a requirement to be "done")
 - 📌 让最终用户参与编写测试用例 (Involve end-users in writing test cases)
 - 📌 要求必须可衡量 (Measurable) 且可测试 (Testable)
-

11. 需求中的冲突、矛盾、不完整性检测 (Conflict, Inconsistency, and Completeness Detection in Requirements)

✅ 冲突检测 (Conflict Detection) :

- 📌 当两个或多个需求相互矛盾 (Two or more requirements contradict each other)
- 📌 示例 (Example):
 - ❌ “系统必须在 2 秒内运行”
 - ❌ “系统必须执行深度分析”
- 📌 解决方案 (Solution): 需求可追溯性 (Traceability) & 冲突矩阵 (Conflict Matrices)

✓ 不一致性检测 (Inconsistency Detection) :

📌 不同部分的需求彼此矛盾 (Different parts of the requirement contradict or don't align)

📌 示例 (Example):

✗ "所有用户必须登录"

✗ "访客用户无需登录即可访问"

📌 技术 (Technique): 交叉检查 (Cross-Checking) 相关需求

✓ 完整性检查 (Completeness Check) :

📌 确保所有必要功能都被描述 (Ensures all required functionalities are described)

📌 无遗漏的约束或条件 (No missing constraints or conditions)

📌 检查表 (Checklist):

✓ 所有 利益相关者 (Stakeholders) 需求是否被满足？

✓ 所有 接口 (Interfaces) 是否被定义？

✓ 歧义检测 (Ambiguity Detection) :

📌 模糊的需求 (Ambiguous Requirement) :

✗ "系统应该很快"

📌 清晰的需求 (Clear Requirement) :

✓ "系统响应时间不得超过 2 秒 (System response time shall not exceed 2 seconds under load)"

12. 可追溯性矩阵 (Traceability Matrix)

📌 需求与以下内容建立映射 (Maps Requirements to):

✓ 利益相关者需求 (Stakeholder Needs)

✓ 设计组件 (Design Components)

✓ 测试用例 (Test Cases)

✓ 优势 (Benefit): 帮助发现缺失或额外的功能 (Helps detect missing or extra functionality)

13. V&V 工具支持 (Tool Support for V&V)

📌 常见工具 (Common Tools):

✓ IBM DOORS

✓ ReqView

✓ JIRA (用于问题跟踪, Issue Tracking)

14. V&V 在敏捷开发中的应用 (V&V in Agile Context)

📌 通过冲刺 (Sprints) 和评审 (Reviews) 进行持续验证 (Continuous Validation)

📌 "完成定义 (Definition of Done, DoD)" 确保验证 (Ensures Verification)

📌 用户故事 (User Stories) 和验收标准 (Acceptance Criteria) 帮助确认需求

15. 总结 (Final Summary)

✓ V&V 确保需求正确且完整 (V&V ensures correct and complete requirements)

✓ 验证 = "我们是否正确地构建?" (Verification = "Did we build it right?")

✓ 确认 = "我们是否构建了正确的产品?" (Validation = "Did we build the right thing?")

✓ 使用多种技术: 检查 (Inspection), 原型 (Prototyping), 测试 (Testing) 🚀

Lecture 5 : 需求演化与管理

一、翻译与精要提炼

✓ 学习目标 (Learning Objectives)

本节课全面介绍了如何在软件开发中管理不断变化的需求。你将理解需求的生命周期，以及在情境变化时如何有效地管理需求。

核心知识点：

- 理解软件需求的演化特性
- 学习应用 需求可追踪性 (Traceability) 技术

- 实践 优先级排序方法（如 MoSCoW 与 Kano 模型）
- 探索通过 基线（Baseline）来管理变化的方法
- 理解 需求重用（Requirement Reuse）的价值

本质：

软件需求是动态的。有效管理需求的演化，才能保障项目的适应性与成功交付。

✅ 需求演化简介（Introduction to Requirements Evolution）

软件需求并非静态不变。它们必须适应环境变化、新的利益相关者需求，以及意外挑战。需求演化是软件生命周期的自然组成部分。

核心知识点：

- 业务与技术驱动因素导致需求变化
- 规划与文档的灵活性是必要的
- 能避免产品功能过时或无关

本质：

灵活应对变化是确保软件持续价值的关键。

✅ 为什么需求会演化（What Causes Requirements to Evolve）

需求演化受多个内外部因素影响，可能显著影响项目范围与交付计划。

核心知识点：

- 来自利益相关者的反馈
- 法规更新
- 市场竞争压力
- 技术限制

本质：

变化源于反馈、法规与市场。敏捷响应，才能保持竞争力。

✅ 实例：移动银行应用（Real-World Example – Mobile Banking App）

初期支持本地交易的应用因业务扩展被要求添加国际转账功能，进而需要包括汇率、合规检查、国际 API。

核心知识点：

- 用户需求随业务增长而改变
- 小功能扩展可引发复杂连锁反应

本质：

需求微调可能牵动系统整体结构。

✅ 需求可追踪性（Requirements Traceability）

可追踪性（Traceability）可帮助从需求起源到部署全过程进行追踪，确保不遗漏任何内容。

核心知识点：

- 支持 影响分析（Impact Analysis）与审计
- 将需求与设计、代码、测试关联
- 提升可验证性与问责制

可追踪性类型（Types of Traceability）：

- 正向可追踪性：需求 → 测试
- 反向可追踪性：测试 → 需求
- 双向可追踪性：双向链接

实例：密码重置（Password Reset）

- 设计：OTP 输入界面
- 代码：`otp_reset.py`

- 测试：TC003
- 部署：版本 1.2

工具：

- IBM DOORS
- Jira 搭配 Xray / Zephyr
- Jama Connect
- ReQtest

本质：

可追踪性是需求质量的保障线。

✓ 需求优先级排序 (Requirements Prioritization)

并非所有需求同等重要，排序有助于聚焦最大价值的交付。

优点：

- 与业务目标对齐
- 有效管理范围
- 改善决策过程

MoSCoW 技术：

- Must have：必须有
- Should have：应当有
- Could have：可以有
- Won't have：当前不考虑

Kano 模型 & 100 美元测试 (100-Dollar Test)：

- 分类：基本需求、性能特征、令人惊喜项
- 分配假想预算，体现价值感知

价值-复杂度矩阵 (Value vs. Complexity Matrix)：

- 高价值低复杂：优先开发
- 高价值高复杂：谨慎规划
- 低价值低复杂：酌情考虑
- 低价值高复杂：尽量避免

银行应用特性示例：

功能	优先级
登录/注册	Must
查看余额	Must
转账	Should
预算计划	Could
加密货币交易	Won't

本质：

有策略地排序，才能把资源用在刀刃上。

✅ 为什么需求会变更（Why Requirements Change）

软件开发中变化是必然的。识别并管理变化极为关键。

原因：

- 新洞察或用户反馈
- 法规合规需求
- 功能缺口
- 新兴技术

本质：

善变是常态，掌控变更是能力。

✅ 变更管理流程（Change Management Process）

步骤：

1. 提交变更请求
2. 影响分析
3. 与相关方审议
4. 批准或驳回
5. 更新 *基线 (Baseline)* 并通知团队

需求基线 (Requirements Baseline)：

冻结某一时刻的需求版本，后续变更须遵循正式流程。

基线优点：

- 提供参考点
- 支持项目追踪
- 控制范围膨胀 (*Scope Creep*)

配置管理 (Configuration Management)：

- 版本控制 (如 *Git*)
- 变更日志
- 发布说明

好处：

- 降低混乱
- 清晰记录演化过程

本质：

| 没有基线的项目，就像无舵的船。

需求重用 (Reusing Requirements)

重用需求提升效率，特别适用于常见模块。

可重用需求示例：

- 登录验证
- 邮件通知
- 基于角色的访问控制 (*Role-based Access*)

优势：

- 节省时间
- 增强可靠性

本质：

好需求值得重复使用，别让轮子重复发明。

全面总结

软件需求不是一次性写完的文档，而是一个“活着”的对象。在整个项目周期内，需求将因外部环境、用户反馈、技术限制而不断演化。

为了有效管理这种演化，我们需要建立需求的**可追踪性 (Traceability)**，明确需求与设计、测试、部署之间的关联；通过 **MoSCoW**、**Kano** 等方法进行合理的优先级排序，使资源投入聚焦高价值产出；在变化不可避免时，通过规范化的 **变更管理流程 (Change Management)** 和 **需求基线 (Baseline)** 控制风险与混乱。同时，针对常见功能，实施 **需求重用 (Reuse)**，可大幅提高效率与一致性。

各章节环环相扣：变化引发需求演化 → 演化需可追踪与排序 → 排序帮助应变与决策 → 再通过流程与基线管控变化 → 最后以重用实现经验沉淀与效率提升。

Lecture 6 高级主题简介 (Introduction to Advanced Topics)

翻译与关键词保留：

随着现代软件系统越来越多地嵌入关键环境（如医疗、交通、金融与国防），需求工程 (Requirements Engineering) 必须不断发展。本章节聚焦四个关键高级领域，以满足对高可靠性、安全性和可维护性的日益增长需求：

- 危害分析与安全关键需求（Hazard Analysis & Safety-Critical Requirements）
- 威胁建模与安全关键需求（Threat Modeling & Security-Critical Requirements）
- 工具支持（Tool Support）
- 开放研究挑战（Open Research Challenges）

核心知识点提炼：

- 软件日益用于高风险行业，需求工程需关注安全性与安全防护。
- 四个高级主题回应实际软件开发中的关键挑战。

本质话：

现代软件系统的复杂性与关键性，推动需求工程迈向更高的安全性与可验证性。

第二部分：危害分析与安全关键需求（Hazard Analysis & Safety-Critical Requirements）

1. 概念说明

翻译：

安全关键系统（Safety-Critical Systems）是指其故障可能导致人身伤害、死亡或重大经济损失的系统，例如心脏起搏器、飞机自动驾驶系统、核电站控制系统。危害分析（Hazard Analysis）是系统性识别潜在故障情境的过程，旨在提前发现并规避可能的灾难性后果。

2. 三种主流技术

技术	方法	适用范围	特点
FMEA（Failure Modes and Effects Analysis）	自底向上	机械、电气系统	量化风险，操作简便
FTA（Fault Tree Analysis）	自顶向下	航空、核能、汽车	可视化因果关系
HAZOP（Hazard and Operability Study）	指导词 + 团队讨论	化工、流程工业	思维全面、团队协作

3. 从危害导出需求

翻译：

一旦识别出危害，便可制定相应的非功能性需求（Non-functional Requirements），例如：“若温度超过80°C，系统应关闭该模块。”

核心知识点提炼：

- 危害分析贯穿于需求制定早期。
- 常见技术如 FMEA、FTA、HAZOP 各有优劣，适用于不同场景。
- 安全需求大多为非功能性，聚焦系统如何应对极端或异常情况。

本质话：

安全需求源于风险预判，是对系统韧性的前置定义。

第三部分：威胁建模与安全关键需求（Threat Modeling & Security-Critical Requirements）

1. 威胁与安全性的区别

翻译：

安全性（Security）已成为所有系统的基本需求。与意外性危害不同，威胁（Threats）是攻击者有意为之。需求工程需主动预判威胁并提出防范机制。

2. 威胁建模步骤

- 确定资产（Assets）
- 分析攻击者（Attacker motives）
- 查找系统漏洞（Vulnerabilities）
- 制定防御需求（Countermeasures）

3. STRIDE 框架（六大威胁分类）

类别	含义	示例	常见防范性需求
Spoofing（伪装）	假冒身份	钓鱼网站窃取密码	实施多因素认证（MFA）

Tampering（篡改）	未经授权地更改数据	注入恶意脚本	数据加密与校验
Repudiation（否认）	行为无法追踪	用户删除数据不承认	审计日志记录与防篡改
Information Disclosure（信息泄露）	敏感数据泄露	浏览器显示调试信息	访问控制
Denial of Service（拒绝服务）	系统无法服务	Bot 攻击网站	请求限流
Elevation of Privilege（权限提升）	非法提升权限	普通用户成为管理员	基于角色的访问控制（RBAC）

核心知识点提炼：

- 安全威胁是可预测的“人为破坏”，需求应反映防御策略。
- STRIDE 提供全面分类，便于系统性设计防御需求。

本质话：

安全需求是防止“蓄意伤害”的系统盾牌。

第四部分：需求工程工具支持（Tool Support in Requirements Engineering）

翻译：

随着系统与需求日益复杂，手动追踪和管理已无法胜任。需求工程工具可以支持需求的收集（gathering）、追踪（traceability）、变更管理（change management）与一致性检查（consistency checking）。

工具分类：

工具类型	代表工具	用途
需求管理工具	IBM DOORS, Jama Connect	编写与组织需求
建模工具	Enterprise Architect	可视化建模
追踪工具	Reqtify	建立需求间关联
形式验证工具	Alloy, SPIN	数学验证系统逻辑

示例工作流程：

- 在 IBM DOORS 定义需求
- 使用 Enterprise Architect 建模交互
- 在 Jira 中连接测试
- 通过 Reqtify 保持全程可追踪性

核心知识点提炼：

- 工具提高效率、协作与可追踪性，尤其适用于安全/高可靠性项目。

本质话：

工具是复杂系统中保障一致性与质量的“支撑架”。

第五部分：开放研究挑战（Open Research Challenges）

翻译：

当今系统越来越智能、自适应、实时，静态的需求文档无法捕捉其动态行为，催生出多个活跃的研究问题：

主要挑战：

- 敏捷开发中的动态需求追踪
- 自然语言歧义自动识别（如“快”、“安全”等模糊词）
- AI 系统的“正确行为”定义
- 社会伦理价值的编码（如公平性、可解释性）
- 自动化需求获取（Elicitation）

创新方向：

- NLP 自动分类需求
- 使用本体（Ontologies）标准化术语
- 群众外包收集公众需求
- 数字孪生（Digital Twins）实时验证需求

核心知识点提炼：

- 新兴问题集中于动态性、模糊性与伦理可解释性。
- 技术手段如 NLP 与仿真模型正成为研究突破口。

本质话：

面向未来的需求工程，需在智能化与道德性之间找到平衡。

总结与章节串联：

本章围绕“软件系统在关键场景下的高要求”展开：

1. 危害分析与安全需求关注系统失效带来的严重后果，强调从预警中提炼非功能性安全需求；
2. 威胁建模与安全需求聚焦抵御人为攻击，从分类出发制定有针对性的安全机制；
3. 工具支持则为复杂项目的全流程追踪与协作提供了可行性保障；
4. 最后，开放研究问题则推动整个学科从“静态规范”向“动态理解”过渡，特别是在 AI 与伦理层面。

全章核心一句话总结：

软件需求工程正从“定义系统做什么”迈向“确保系统安全地做得对、并始终做对”。